Coding Questions

Problem 1: Payment System Fee Calculation

Task

A payment processing system handles various transaction types. Each payment method applies a different processing fee. For every transaction, calculate the final amount after applying the applicable fee and display the adjusted amount for each transaction followed by the total amount processed.
Input The first line contains an integer N, the number of transactions. Each of the next N lines contains a payment type and transaction amount.

- CARD Amount
- WALLET Amount
- BANKTRANSFER Amount

Output For each transaction, display PaymentType: AdjustedAmount, formatted to two decimal places. Finally, display Total: GrandTotal, also formatted to two decimal places.

Business Rules

- Card transactions have a 2% processing fee.
- Wallet transactions have a 1% processing fee.
- Bank Transfers have no processing fee.

Constraints

- 1 <= N <= 1000
- 1 <= Amount <= 1000000

Sample Input
3

CARD 1000

WALLET 500

BANKTRANSFER 2000

Expected Output
CARD: 1020.00

WALLET: 505.00

BANKTRANSFER: 2000.00

Total: 3525.00

Instructions Implement the solution so that each payment type can apply its own calculation behavior while the main processing flow handles all transactions uniformly.

Instructions

Consider how to model different payment methods that share a common operation (calculating the final amount) but have distinct fee calculation logic. Focus on how a central system can process a collection of different payment types without explicit type checking.

Problem 2: Library Item Due Date Calculator

Task

A library manages different types of items (e.g., Books, DVDs, Magazines). Each item type has a specific borrowing duration. The system needs to process a list of borrowed items and calculate their respective due dates from the current date.

Input The first line contains an integer N, the number of borrowed items. Each of the next N lines contains ItemType ItemTitle.

- BOOK "The Great Gatsby"
- DVD "Inception"
- MAGAZINE "National Geographic Jan 2023"

Output For each borrowed item, display ItemTitle: DueDate. The DueDate should be formatted as YYYY-MM-DD.

Business Rules

- Books have a standard borrowing period of 14 days.
- DVDs have a standard borrowing period of 7 days.
- Magazines have a standard borrowing period of 3 days.
- The due date is calculated relative to the "current date". For simplicity, assume the current date is 2023-10-26.

Constraints

- 1 <= N <= 100
- ItemTitle is a string.

Sample Input
3

BOOK "1984"

DVD "The Matrix"

MAGAZINE "Forbes Issue 500"

Expected Output
1984: 2023-11-09

The Matrix: 2023-11-02

Forbes Issue 500: 2023-10-29

Instructions Implement a solution that allows different library item types to determine their due date based on their specific rules, while the main processing logic can handle all items uniformly. You may use Python's datetime module for date calculations.

Instructions

Consider how to model different library item types that share common attributes but have distinct borrowing rules. Focus on how a central system can calculate due dates for a collection of various item types without explicit type checking.

Problem 3: Delivery Fee Calculator

Task

A delivery service offers different types of delivery (e.g., Standard, Express, International). Each delivery type has unique rules for calculating its fee based on weight and distance. The system needs to calculate the total fee for a list of delivery requests.

Input The first line contains an integer N, the number of delivery requests. Each of the next N lines contains DeliveryType Weight Distance [AdditionalParam]

- STANDARD Weight Distance
- EXPRESS Weight Distance
- INTERNATIONAL Weight Distance CustomsFee

Output For each delivery, display DeliveryType: CalculatedFee, formatted to two decimal places. Finally, display Total: GrandTotalFee, also formatted to two decimal places.

Business Rules

- Standard Delivery: Base fee $5, plus $0.50 per kg, plus $0.10 per km.
- Express Delivery: Base fee $15, plus $1.00 per kg, plus $0.20 per km.
- International Delivery: Base fee $25, plus $2.00 per kg, plus $0.50 per km, plus an additional CustomsFee.

Constraints

- 1 <= N <= 100
- Weight, Distance, CustomsFee are positive numbers.

Sample Input
3

STANDARD 10 50

EXPRESS 5 20

INTERNATIONAL 20 100 30

Expected Output
STANDARD: 15.00

EXPRESS: 29.00

INTERNATIONAL: 155.00

Total: 199.00

Instructions Implement a solution such that each delivery type calculates its fee according to its specific rules, while the main system can process all delivery requests uniformly.

Instructions

Consider how to represent different delivery types that share a common operation (calculating a fee) but have distinct pricing logic. Focus on how a central system can calculate fees for a collection of different delivery types without explicit type

checking.

Problem 4: Examination Question Grader

Task

An examination system needs to evaluate answers for different types of questions (e.g., Multiple Choice, True/False, Essay). Each question type has a specific grading logic. The system must process a list of questions with provided answers and determine the score for each, then calculate the total score.

Input The first line contains an integer N, the number of questions. Each of the next N lines contains QuestionType QuestionText CorrectAnswer StudentAnswer [Points]

- MCQ "What is 2+2?" "4" "4" 10
- TF "Water boils at 100C?" "True" "True" 5
- ESSAY "Discuss OOP principles" "Polymorphism, Inheritance, Encapsulation" "I talked about Polymorphism and Inheritance." 20

Output For each question, display QuestionType: Score, formatted to two decimal places. Finally, display Total Score: OverallScore, also formatted to two decimal places.

Business Rules

- Multiple Choice Question (MCQ): Full points if StudentAnswer exactly matches CorrectAnswer.
- True/False Question (TF): Full points if StudentAnswer exactly matches CorrectAnswer.
- Essay Question (ESSAY): Partial points. If StudentAnswer contains at least two of the keywords from CorrectAnswer (comma-separated, case-insensitive), award 75% of Points. If one keyword, award 50%. Otherwise, 0 points.
- For simplicity, assume CorrectAnswer for Essay questions is a comma-separated string of keywords.

Constraints

- 1 <= N <= 100
- Points are positive integers.
- All string comparisons are case-insensitive for essay keywords.

Sample Input
4

MCQ "What is the capital of France?" "Paris" "Paris" 10

TF "The Earth is flat?" "False" "True" 5

ESSAY "Name two primary OOP principles." "Inheritance, Polymorphism, Encapsulation" "Polymorphism is one." 20

ESSAY "Describe abstraction and composition." "Abstraction, Composition" "I talked about abstraction." 15

Expected Output
MCQ: 10.00

TF: 0.00

ESSAY: 10.00

ESSAY: 7.50

Total Score: 27.50

Instructions Develop a system where different question types can evaluate answers based on their specific grading criteria, allowing a central grader to process all questions uniformly. Your solution should parse the input, grade each question, and then print the individual and total scores as specified.

Instructions

Consider how to model different examination question types that share a common operation (evaluating an answer) but have distinct grading logic. Focus on how a central grading system can process a collection of various question types without explicit type checking.

Problem 5: Public Transport Fare Calculator

Task

A public transport system calculates fares based on the type of transport (e.g., Bus, Train, Metro) and distance traveled. Each transport type has a different fare structure. The system needs to calculate the fare for a list of journeys and display the individual and total fares.

Input The first line contains an integer N, the number of journeys. Each of the next N lines contains TransportType Distance [AdditionalParam]

- BUS Distance
- TRAIN Distance
- METRO Distance PeakHourFactor

Output For each journey, display TransportType: CalculatedFare, formatted to two decimal places. Finally, display Total: GrandTotalFare, also formatted to two decimal places.

Business Rules

- Bus: Base fare $2, plus $0.10 per km. Max fare $10.
- Train: Base fare $3, plus $0.15 per km.
- Metro: Base fare $1.50, plus $0.20 per km, multiplied by a PeakHourFactor.

Constraints

- 1 <= N <= 100
- Distance is a positive number.
- PeakHourFactor is a float between 1.0 and 2.0 (inclusive).

Sample Input
3

BUS 15

TRAIN 50

METRO 10 1.5

Expected Output
BUS: 3.50

TRAIN: 10.50

METRO: 5.25

Total: 19.25

Instructions Implement a system where different transport types can calculate their fares according to their specific rules, allowing a central system to process all journey requests uniformly.

Instructions

Consider how to model different transport types that share a common operation (calculating a fare) but have distinct pricing logic. Focus on how a central system can calculate fares for a collection of different transport types without explicit type checking.

Quiz Questions

Question 1

A system processes Document objects. It initially had PDFDocument and WordDocument classes, both inheriting from Document and implementing a render() method. A developer observes repeated if-else if blocks checking the document type before calling render(). What is the most appropriate OOP principle to apply to eliminate this conditional logic?

A Encapsulation
B Method overloading
C Polymorphism
D Abstraction

Question 2

Which of the following scenarios represent a genuine 'is-a' relationship, making inheritance a suitable design choice? (Select all that apply)

A A `Car` is a `Vehicle`.
B A `Rectangle` is a `Shape`.
C A `DatabaseConnection` is a `NetworkResource`.
D A `HelperUtility` contains a `Calculator`.

Question 3

Consider a Vehicle base class with a method startEngine() and two derived classes, Car and Motorcycle, both overriding startEngine() with their specific engine start sounds. If you have a list of Vehicle objects, some being Car instances and some Motorcycle instances, and you iterate through this list calling startEngine() on each object, what mechanism ensures the correct startEngine() implementation is called for each vehicle type?

A Static method binding
B Compile-time polymorphism
C Runtime method dispatch
D Method overloading

Question 4

A Shape base class has a method calculateArea(). Circle and Rectangle are derived classes, each with their own implementation of calculateArea(). If a program stores various Shape objects in a list and then calls calculateArea() on each, what concept is being demonstrated?

A Method hiding
B Method overloading
CI nheritance-based polymorphism
D Composition

Question 5

A software component is designed to manage various LibraryItem types like Book and DVD. Both Book and DVD inherit from LibraryItem. The LibraryItem class has a getLoanPeriod() method. The Book class extends this method by adding a special rule for new releases, while the DVD class sets a fixed loan period. Which of the following statements accurately describe the behavior in this system? (Select all that apply)

A `getLoanPeriod()` in `Book` is an example of extending inherited behavior.
B `getLoanPeriod()` in `DVD` is an example of shared behavior.
C The `LibraryItem` class defines the common behavior for all library items.
D A `DVD` object can be processed as a `LibraryItem` through a common reference.

Question 6

Consider a base class Animal with a method makeSound() and derived classes Dog and Cat, which override makeSound() to produce 'Woof' and 'Meow' respectively. Which of the following statements about the behavior of these classes are correct? (Select all that apply)

A If a `Dog` object is referred to by an `Animal` reference, calling `makeSound()` will execute `Dog`'s `makeSound()`.
B The `makeSound()` method in `Animal` is an example of specialized behavior.
C The `makeSound()` method in `Dog` is an example of overridden behavior.
D New behavior can only be added to derived classes if it is also declared in the base class.
Question 7

A PaymentProcessor system processes CardPayment and BankTransferPayment objects. Both CardPayment and BankTransferPayment derive from a Payment base class and override a calculateFee() method. If the PaymentProcessor maintains a collection of Payment references, what is the primary benefit of this design when adding a new WalletPayment type?

A It reduces the need for new data members in the base class.
B It ensures all payment types use the same fee calculation logic.
C It allows adding `WalletPayment` without modifying the existing `PaymentProcessor`'s iteration logic.
D It enhances the security of payment transactions.
Question 8

What is the primary reason why using inheritance solely for superficial code reuse between unrelated classes is generally considered an inappropriate design choice?

A It leads to tighter coupling and incorrect 'is-a' relationships, making the design rigid.
B It always results in slower program execution due to increased overhead.
C It complicates debugging by creating too many small classes.
D It prevents the use of interfaces in the system.
Question 9

A Notification base class has a send() method. Derived classes EmailNotification, SMSNotification, and PushNotification each override send() to implement channel-specific delivery logic. What are the advantages of using inheritance and polymorphism in this notification system design? (Select all that apply)

A It allows a generic `NotificationSender` to send various types of notifications without knowing their concrete types.
B It centralizes all notification logic into a single `Notification` class, reducing code duplication across derived classes.
C It simplifies the process of adding a new notification channel, like `InAppNotification`, without altering existing sender logic.
D It ensures that all notification types use the exact same `send()` implementation, regardless of the channel.

Question 10

Which of the following are benefits of using polymorphic collections (e.g., a list of base class references holding derived class objects)? (Select all that apply)

A It simplifies iterating over diverse but related objects.
B It allows for uniform processing of objects with specialized behavior.
C It reduces the need for explicit type casting in common processing loops.
D It makes it harder to add new derived types to the system.

Concept Question

Question 1
Explain how inheritance enables specialized classes to reuse existing behavior from a base class and simultaneously extend or modify that behavior to suit their specific needs. Provide a concrete business example.

Question 2
Describe the 'is-a' relationship in the context of object-oriented programming. Explain why correctly identifying a genuine 'is-a' relationship is crucial when deciding to use inheritance in a class design.

Question 3
Define method overriding. Using a realistic business scenario, such as different types of Employee objects calculating their salary differently, illustrate how method overriding allows derived classes to provide their own specific implementations for a method defined in their base class.

Question 4
Explain the concept of runtime polymorphism (dynamic dispatch) when working with common base-type references. How does the system determine which specific method implementation to execute when a method is called on a base-type reference that actually points to a derived class object?

Question 5
Describe how polymorphic collections work. Provide an example of a collection (e.g., a list or array) that holds objects of a common base type, but where the actual objects are instances of different derived types. Explain the advantages of using such a collection for processing diverse but related objects.

Question 6
Compare and contrast solving problems using polymorphism (specifically, method overriding with a common base type) versus using repeated type-based conditional logic (e.g., if-else if or switch statements). Explain why polymorphism is generally preferred for managing varying behavior among related objects.

Question 7
Explain how the principles of inheritance and polymorphism allow for the addition of new derived types to an existing design with minimal or no changes to the common processing logic that operates on the base type. Illustrate this with an example where a new PaymentMethod is introduced into a payment processing system.

Question 8
Distinguish between inherited behavior and overridden behavior in object-oriented programming. When would a derived class typically inherit a method, and when would it typically override a method? Provide examples for each.

Question 9
Explain why using inheritance solely for the sake of code reuse, particularly when there is no genuine 'is-a' relationship between the classes, can be an inappropriate design choice. Discuss the potential negative consequences of such a design.

Question 10
Analyze a scenario, such as a VehicleRental system dealing with CarRental and TruckRental, to identify both the common behaviors shared across all rentals and the specialized behaviors unique to each type of rental. Explain how inheritance can be used to model this distinction effectively.